

Πανεπιστήμιο Κύπρου
University of Cyprus

Accelerating Deep Neural Networks with FPGAs

Stylianos Mikeos

Supervisor: Dr. Theocharis Theocharides

December 2025

Table of Contents

Table of Contents 2
List of Acronyms / Abbreviations 4
1. Introduction 5
1.1. Motivation and Thesis Contributions 6
1.2. Organizational Outline 8
2. Background and Related Work 9
2.1. Background Work 9
2.1.1. Convolutional Neural Networks and Inference Workload 9
2.1.2. Evaluation Metrics for Embedded Inference 9
2.1.3. Why FPGAs for CNN Inference 10
2.1.4. Numerical Precision and Quantization 10
2.1.5. Approaches to Implementing CNNs on FPGAs 11
2.1.5.1. Manual RTL Design (VHDL/Verilog) 11
2.1.5.2. High-Level Synthesis (HLS) using C/C++ 11
2.1.5.3. Vendor DPU Frameworks (Vitis AI) 11
2.1.5.4. Automated CNN-to-FPGA Toolflows (fpgaConvNet, FINN) 12
2.1.6. Model Interoperability with ONNX 12
2.2. Summary and Research Gap 12
3. Methodology and Thesis Contributions 14
3.1. End-to-End Workflow Overview 14
3.2. Deployment Targets and Practical Constraints (ZedBoard vs KCU105) 15
3.3. Key Contributions 15
3.3.1. Reproducible Notebook-Based Pipeline (Key Contribution) 15
3.3.2. Extending fpgaConvNet Platform Support 16
3.3.3. Expanding Model Coverage Beyond the Default Tutorial 16
3.4. Case Study A: Tutorial Baseline on ZedBoard 16
3.5. Summary 17
4. Case Studies 17
4.1. Experimental Context and Targets 17
4.2. ONNX-Based Deployment and Validation on ZedBoard 18
4.2.1. Model preparation and ONNX export 18
4.2.2. Software reference inference with ONNX Runtime 18
4.2.3. fpgaConvNet mapping and Vivado build 19
4.3. Host–FPGA Interface and Output Verification (ZedBoard) 19
4.4. Custom End-to-End Debug Configuration (ZedBoard) 19

4.5. Models and Datasets Used in the Case Studies 20
4.6. Reproducibility and Artifacts 20
4.7. Summary 20
5. Evaluation 21
5.1. Evaluation Objectives and Metrics 21
5.2. ZedBoard: Implemented Prototype Measurements 21
5.3. ZedBoard: Output Verification (CPU vs FPGA) 23
5.4. KCU105: Multi-Model Evaluation Setup 23
5.5. KCU105: Partitioning Behaviour 24
5.6. KCU105: Estimated Resource Utilization and Feasibility 24
5.7. KCU105: Estimated Latency and Throughput Trends 26
5.8. Summary of Findings 29
6. Conclusions 30
6.1. Summary of Thesis Outcomes 30
6.1.1. Thesis Contributions 30
6.1.2. Key Findings from ZedBoard On-Board Measurements 31
6.1.3. Insights from KCU105-Based Multi-Model Evaluation and Design-Space Exploration 31
6.1.4. Limitations 32
6.1.5. Future Work 32
7. References 34

List of Acronyms / Abbreviations

BN — Batch Normalization
BRAM — Block Random Access Memory (FPGA on-chip memory)
CPU — Central Processing Unit
CNN — Convolutional Neural Network
DNN — Deep Neural Network
DMA — Direct Memory Access
DPU — Deep-learning Processing Unit (vendor accelerator IP)
DSP — Digital Signal Processing (FPGA DSP slice / multiplier-accumulator block)
DSE — Design-Space Exploration
FF — Flip-Flop (FPGA register resource)
FPGA — Field-Programmable Gate Array
GPU — Graphics Processing Unit
HLS — High-Level Synthesis
IoT — Internet of Things
LUT — Look-Up Table (FPGA logic resource)
MAC — Multiply–Accumulate
MNIST — Modified National Institute of Standards and Technology (handwritten digit dataset)
MPSoC — Multiprocessor System-on-Chip
ONNX — Open Neural Network Exchange
PL — Programmable Logic (FPGA fabric, Zynq terminology)
PS — Processing System (ARM processor subsystem, Zynq terminology)
RTL — Register-Transfer Level
SDF — Synchronous Dataflow
SoM — System-on-Module
UART — Universal Asynchronous Receiver–Transmitter
UAV — Unmanned Aerial Vehicle
VHDL — VHSIC Hardware Description Language
VHSIC — Very High Speed Integrated Circuit

1. Introduction

Deep learning, and in particular deep convolutional neural networks (CNNs), has become the dominant approach for a wide range of perception and decision-making tasks, from image classification and object detection to speech and natural language processing. Their success, however, comes at the cost of substantial computational and memory requirements. Modern CNNs often require billions of operations per input and large parameter sets, making deployment on embedded or edge platforms particularly challenging.

General-purpose central processing units (CPUs) typically fail to deliver the required performance within realistic power and cost budgets. Graphics processing units (GPUs) provide massive parallelism and are widely used for training and inference in data centres, but their power consumption and form factor are often prohibitive for embedded systems such as drones, Internet of Things (IoT) devices or low-cost development boards. This has led to increasing interest in field-programmable gate arrays (FPGAs) as accelerators for deep neural network (DNN) inference. FPGAs offer a combination of reconfigurability, fine-grained parallelism and relatively low power consumption, enabling custom accelerator architectures tailored to a specific model and application [1], [6].

At the same time, there is growing research interest in dynamic neural networks and adaptive inference, where the network's depth, width or input resolution can change at runtime depending on the input or system constraints. For example, in multi-altitude unmanned aerial vehicle (UAV) object detection, different altitudes favour different CNN configurations in terms of accuracy–efficiency trade-offs [2]. Dynamic CNNs, such as early-exit networks or architectures with altitude-dependent parameters, can significantly reduce computation while preserving accuracy by adapting to the difficulty of each input [2], [3]. From a hardware perspective, this naturally connects to the reconfigurability of FPGAs and raises questions about how to support such dynamic behaviour efficiently on resource-constrained devices.

However, in practice, building an end-to-end toolflow that takes a trained network and maps it to an embedded FPGA is still complex. The designer must navigate device-specific toolchains, licensing constraints, and low-level details such as memory interfaces, clocking and input/output standards. High-level synthesis (HLS) tools and vendor deployment frameworks can reduce development effort, but in practice they still require significant toolchain expertise and are often constrained by supported board/device targets and reference workflows. This can make portability and debugging difficult when moving outside well-documented example designs, motivating the use of a more flexible mapping toolflow such as fpgaConvNet for experimentation across different FPGA platforms [10], [22], [23].

In this context, fpgaConvNet provides an attractive alternative: it is a research-grade toolflow designed specifically for mapping CNNs to FPGAs [1]. Given a pre-trained CNN model, a device description and high-level performance targets, fpgaConvNet uses a synchronous dataflow (SDF) formulation and analytical models to automatically explore the architectural design space and generate hardware accelerators optimised for throughput, latency or multi-objective criteria [1]. Crucially for this thesis, fpgaConvNet supports taking models from modern deep learning frameworks (via extensions to support the Open Neural Network Exchange, ONNX) [4] and can target multiple FPGA platforms through board/device descriptions. In our work, we used the ZedBoard for on-board emulation and measured validation, and we also targeted a higher-resource KCU105 configuration to evaluate and compare multiple CNN models more broadly under a larger FPGA resource budget [1], [6].

The overarching goal of this thesis is therefore to systematically investigate the acceleration of deep neural networks on FPGAs using fpgaConvNet, with a focus on:

- Building a robust, reproducible end-to-end flow from trained models (exported in ONNX format) to FPGA accelerators, and
- Evaluating multiple CNN models using two complementary evaluation settings:
    - Emulated/measured results on an embedded FPGA platform (ZedBoard), and
    - Estimated latency/throughput/resource feasibility trends via fpgaConvNet targeting a larger KCU105 configuration

Although dynamic and reconfigurable neural network architectures are an important long-term motivation, the thesis concentrates on static inference accelerators, using fpgaConvNet as the main vehicle. Dynamic CNNs and runtime reconfiguration are treated as related work and future extensions, informed by recent research on dynamic DNNs and altitude-adaptive object detection [2], [3].

1.1. Motivation and Thesis Contributions

The motivation for this work is twofold.

First, there is a practical deployment challenge. Even relatively small CNNs that are commonly used in academic benchmarks, such as models trained on the CIFAR-10 and CIFAR-100 datasets [7], can be difficult to deploy efficiently on low-cost embedded platforms. Designers must balance latency, accuracy, resource usage and power under strict device constraints. A systematic and documented flow from high-level models to FPGA implementations is valuable both for research and for practitioners.

Second, there is a need to bridge research toolflows and real boards. Tools like fpgaConvNet demonstrate impressive performance-per-watt on paper and in simulation [1], but there is often a gap between "toolflow papers" and a student or engineer actually taking a model, mapping it, and running it on a specific board such as the ZedBoard. Closing this gap, including the many small but critical steps and pitfalls is an important contribution in itself. At the same time, it is also important to understand how conclusions change when moving from a low-resource embedded board to a higher-resource FPGA target; therefore, this thesis also uses a KCU105-targeted configuration to evaluate a wider set of models and observe how resource budgets affect feasibility and performance trends [1], [6].

Based on these motivations, the main contributions of this thesis are:

- **C1. Comparative exploration of FPGA-based CNN toolflows.**
  Early in the project, multiple approaches were explored (Vivado/Vitis HLS on a Kintex UltraScale KCU105 board, Vitis AI on a Kria KV260 SoM, and fpgaConvNet on the ZedBoard). This exploration is analysed and distilled into a set of practical lessons on what works, what does not, and why, under real constraints of licensing, documentation and setup overhead.

- **C2. End-to-end fpgaConvNet-based flow for ONNX models on ZedBoard.**
  The thesis develops and documents a complete workflow that starts from trained CNN models (initially CIFAR-10/100 models [7], [8]), converts them to ONNX, processes them using the fpgaConvNet toolflow and finally generates hardware accelerators that can run on the ZedBoard [1]. Custom Jupyter notebooks unify and extend fpgaConvNet tutorials into a single reproducible pipeline for parsing, optimisation, high-level synthesis and bitstream generation.

- **C3. Case-study evaluation across CNN models and FPGA targets (and platform-library extension).**
  This thesis includes an empirical evaluation of multiple CNN architectures under fpgaConvNet using two FPGA target configurations: **ZedBoard** (used for emulated / on-board validation) and **KCU105** (used to evaluate how different models scale on a higher-resource FPGA through tool-based estimates). In addition, as a practical contribution, the fpgaConvNet **platform library was extended** to expand supported targets beyond the upstream set by adding new platform/device descriptions for: **AV Arty Z7-20, KC705, KCU105, KCU116, KCU1500, SP701, VCU108, VCU110, VCU118, ZC702, ZCU106, ZCU111, ZCU1275, and ZCU1285**. This extension improves the portability and reproducibility of the workflow by enabling consistent cross-board exploration even when physical hardware access is limited. [1], [6].

- **C4. Discussion of implications for dynamic and reconfigurable CNNs on FPGAs.** While the implemented accelerators are static, the results and practical experience are interpreted in light of recent work on dynamic DNNs and adaptive CNNs for UAVs [2], [3]. This leads to recommendations and concrete ideas for how future work could combine fpgaConvNet-style mappings with runtime adaptability and (partial) reconfiguration, enabling dynamic parameter changes or profile selection on FPGAs, in the spirit of ONNX-based adaptive flows [4].

Overall, the thesis contributes both a practical reference for using fpgaConvNet on a real embedded platform and an experimental study that characterises the behaviour of CNN accelerators under different design choices across two FPGA resource budgets (ZedBoard and KCU105).

1.2. Organizational Outline

The remainder of this thesis is structured as follows.

- **Chapter 2 – Background and Related Work** provides the necessary theoretical and technical background. It reviews deep neural networks and convolutional neural networks, hardware platforms for DNN inference with a focus on FPGAs, FPGA-based CNN toolflows (including fpgaConvNet [1] and ONNX-based flows [4]), and recent work on dynamic neural networks and multi-altitude UAV object detection [2], [3]. It concludes by identifying the research gap addressed by this thesis.
- **Chapter 3 – The fpgaConvNet Framework** presents the fpgaConvNet toolflow in detail. The chapter explains how CNNs are represented using synchronous dataflow graphs, how hardware design parameters are explored and optimised, and how platform-specific configurations are defined [1]. The integration of fpgaConvNet with the Xilinx Vivado toolchain and the ZedBoard platform is described, establishing the basis for the subsequent case studies.
- **Chapter 4 – Case Studies** describes the experimental case studies. It first briefly summarises initial attempts using other FPGA toolflows and boards, highlighting the challenges encountered. It then focuses on the main fpgaConvNet-based case studies, describing the selected CNN models, datasets (CIFAR-10/100 [7], [8]), quantisation settings and the two evaluation targets used in this thesis (ZedBoard for emulated/measured results and KCU105-targeted configuration for broader cross-model evaluation).
- **Chapter 5 – Evaluation** presents and analyses the experimental results. It outlines the evaluation methodology and measurement setup, then reports latency, throughput, FPGA resource utilisation and accuracy results for each case study. Results include measured/emulated ZedBoard performance as well as KCU105-targeted comparative results across multiple models. The chapter compares the different implementations, discusses observed trade-offs and relates the findings to design-space exploration using fpgaConvNet [1], [6].
- **Chapter 6 – Conclusions and Future Work** summarises the main findings and contributions of this thesis. It reflects on the effectiveness of fpgaConvNet for accelerating CNNs on embedded FPGAs, highlights limitations of the current approach and outlines directions for future work, including possibilities for dynamic and altitude-aware CNN deployment on reconfigurable hardware platforms [2]–[4].

- **Chapter 7 – References** lists all bibliographic sources cited throughout the thesis.

2. Background and Related Work

This chapter provides the background needed to understand FPGA-based acceleration of convolutional neural network (CNN) inference and motivates the methodology used later in the thesis. It first reviews CNN inference workloads and the evaluation metrics typically used for embedded deployments. It then introduces key FPGA acceleration principles, discusses numerical precision and quantization, and surveys the main approaches for implementing CNNs on FPGAs: manual RTL design, high-level synthesis (HLS), vendor deployment stacks (e.g., Vitis AI), and research toolflows such as fpgaConvNet and FINN. Finally, it describes the role of ONNX as an interoperability layer for portable deployment pipelines and closes with a summary and research gap.

2.1. Background Work

2.1.1. Convolutional Neural Networks and Inference Workload

Convolutional neural networks (CNNs) are widely used for visual recognition tasks such as image classification and object detection. A CNN is composed of layers that apply convolution operations, nonlinear activations, and often normalization and pooling. At inference time, the model executes a feed-forward computation that transforms an input image into a set of output scores (logits) and a final predicted class.

From a systems perspective, CNN inference is dominated by two resource demands:

1. **Compute intensity:** Convolution layers perform a large number of multiply–accumulate (MAC) operations. Even moderately sized CNNs can require millions to billions of operations per input depending on resolution and architecture.
2. **Memory and data movement:** CNNs require access to weights (parameters) and intermediate activations. The energy and latency cost of moving data often becomes as important as raw compute throughput, especially on embedded platforms.

CNN architectures differ significantly in their compute and memory footprints. For example, residual networks (ResNets) introduce skip connections that improve training and accuracy scaling, while MobileNet-style designs reduce cost using depth wise separable convolutions and channel-width multipliers [18]. These architectural differences strongly affect hardware mapping feasibility and performance.

2.1.2. Evaluation Metrics for Embedded Inference

Embedded inference is constrained by limited power, cost, and compute capability. As a result, evaluation must consider multiple metrics simultaneously:

- **Latency:** The time to produce an output for one input. For interactive or real-time use, latency is often the primary metric.

- **Throughput:** The number of inferences per second. Throughput is important when processing streams or batches of inputs.
- **Accuracy:** The model's predictive performance. Hardware-oriented optimizations (e.g., quantization) can affect accuracy.
- **Resource utilization:** For FPGAs, utilization of LUTs, FFs, BRAMs, and DSP blocks indicates feasibility and headroom for larger designs.
- **Energy efficiency / performance-per-watt:** Particularly relevant for edge devices and battery-powered systems.

In practice, these metrics trade off against each other. Reducing latency may require more parallel hardware and thus more resource usage; increasing throughput may require deeper pipelining and buffering; reducing precision may improve performance but risks accuracy loss.

2.1.3. Why FPGAs for CNN Inference

Field-programmable gate arrays (FPGAs) are attractive for embedded inference because they combine **reconfigurability** with **fine-grained parallelism**. Unlike CPUs, which execute instructions sequentially with limited vectorization, and unlike GPUs, which provide massive parallelism but often at higher power and with a fixed execution model, FPGAs allow the designer (or toolflow) to build custom datapaths tailored to the target network and constraints.

Key FPGA advantages for CNN inference include:

- **Custom parallelism:** Convolution can be parallelized across channels, filters, and spatial dimensions.
- **Pipelining and streaming:** Dataflow-style accelerators can stream activations through a sequence of compute stages, reducing round-trips to external memory and improving throughput.
- **Precision flexibility:** FPGAs can efficiently implement fixed-point arithmetic and mixed-precision designs.
- **Reconfigurability:** Hardware can be adapted to different models or operating points (even if many practical deployments remain static at runtime).

However, FPGA deployment also introduces practical complexity: toolchains (synthesis, place-and-route, bitstreams), platform-specific integration (I/O, clocks, memory), and the need to balance resources carefully.

2.1.4. Numerical Precision and Quantization

CNN inference does not always require full floating-point precision. Quantization reduces numerical precision (e.g., from FP32 to FP16 or fixed-point), improving efficiency in compute and memory bandwidth. FPGA accelerators often favour fixed-point representations because they map well to LUT/DSP resources and allow more parallel MAC units within the same resource budget.

Quantization introduces several considerations:

- **Arithmetic mapping:** Fixed-point multiply and accumulate must manage scaling and overflow.
- **Activation ranges:** Layer outputs may require re-scaling between stages.

- **Accuracy impact:** Lower precision can reduce accuracy depending on the network and calibration.

Frameworks and toolflows differ in how they support quantization. Some prioritize extreme quantization (binary/ternary), while others support moderate precision (e.g., 8–16-bit fixed-point). The key point for this thesis is that FPGA feasibility and performance are strongly linked to precision choices, and correctness validation must account for expected numerical differences between software reference and FPGA execution.

2.1.5. Approaches to Implementing CNNs on FPGAs

There are multiple pathways to map CNN inference onto FPGA hardware. They mainly differ in required effort, portability, and flexibility.

2.1.5.1. Manual RTL Design (VHDL/Verilog)

The most direct approach is to implement CNN operators (convolution, pooling, activation) in RTL using VHDL or Verilog. This provides maximum control over microarchitecture, timing, and resource usage, and it can yield highly optimized designs.

However, manual RTL design is time-consuming and error-prone for large CNNs. It also makes it difficult to iterate quickly across many models or design points, which limits its suitability for rapid comparative evaluation.

2.1.5.2. High-Level Synthesis (HLS) using C/C++

High-Level Synthesis (HLS) raises abstraction by compiling C/C++ (or similar) descriptions into RTL. HLS can accelerate development by allowing algorithmic code to be transformed into hardware pipelines with pragmas guiding unrolling, pipelining, and memory partitioning [22].

HLS is a strong middle ground: it can produce efficient accelerators, but it still typically requires substantial hardware awareness and careful optimization. The developer must manage data movement, memory interfaces, and performance bottlenecks, and must integrate the output with a larger FPGA system.

2.1.5.3. Vendor DPU Frameworks (Vitis AI)

Vendor stacks such as Vitis AI provide end-to-end deployment flows centred on a pre-built “DPU” (Deep Processing Unit) architecture. The typical flow is: quantize → compile → deploy to a supported platform with a runtime. This approach can be highly practical when the target board is supported and the deployment fits the intended workflow.

In practice, two constraints matter for thesis-scale experimentation:

1. **Platform support limitations:** DPU-centric flows target specific device families and do not cover all boards (for example, Zynq-7000 class devices such as ZedBoard are generally outside the main supported scope) [23].
2. **Workflow complexity and rigidity:** The toolchain involves multiple components (model compiler, runtime, DPU integration, version alignment). This can be difficult to adapt for rapid cross-model exploration when working outside standard reference designs.

As a result, while Vitis AI is a strong option for “supported platform → deploy an application”, it can be less suitable when the goal is to explore many CNN architectures across different FPGA resource budgets and compare feasibility trends.

2.1.5.4. Automated CNN-to-FPGA Toolflows (fpgaConvNet, FINN)

Research toolflows aim to reduce manual design effort by automatically mapping CNNs to FPGA accelerators.

fpgaConvNet is designed to map diverse CNNs onto FPGA platforms using a dataflow formulation and analytical design-space exploration [1]. Given a CNN model and a device/platform description, fpgaConvNet explores candidate architectures and estimates key metrics such as resource utilization, throughput, and latency. This is particularly useful for comparative evaluation because it enables systematic exploration rather than ad hoc manual tuning.

FINN is another representative framework that focuses on highly quantized inference (including binarized neural networks) to maximize throughput and efficiency [20]. While FINN's primary emphasis is on very low precision, it demonstrates an important broader concept: numerical precision is a first-class design parameter for FPGA inference, and hardware mappings can be optimized around that choice.

In this thesis, fpgaConvNet is used as the main toolflow because it supports multi-model exploration and board targeting via platform descriptions, aligning with the thesis goal of evaluating different CNN architectures under different resource budgets [1].

2.1.6. Model Interoperability with ONNX

Practical deployment usually begins in a training framework (commonly PyTorch) and then transitions to a portable representation for inference. Open Neural Network Exchange (ONNX) is widely used as an interchange format that enables portability across frameworks and deployment backends [11]–[13].

A common workflow for portable deployment is:

1. Train or obtain a model in PyTorch,
2. Export the model to ONNX using the PyTorch ONNX exporter [12], [13],
3. Validate the exported model in software (e.g., ONNX Runtime) to establish a reference output [11],
4. Feed the ONNX model into a hardware mapping toolflow or deployment pipeline [1], [4].

In this thesis, ONNX is essential for reproducibility and correctness validation: ONNX Runtime provides a CPU reference inference path used to compare outputs (logits and predicted class) against FPGA-generated outputs, while the ONNX graph serves as the model entry point for the mapping pipeline [11]–[13].

2.2. Summary and Research Gap

This chapter reviewed the main technical background required to understand CNN deployment on FPGAs and motivated the workflow choices made in this thesis. CNN inference is a compute- and memory-intensive workload, where performance is typically evaluated using latency, throughput, resource utilization, and energy efficiency. FPGAs are attractive for embedded inference because they can provide strong performance-per-watt through deep pipelining, parallelism, and customized datapaths, but achieving these benefits requires careful mapping decisions and practical tool support.

A key theme in the literature is the trade-off between control and productivity in FPGA implementation approaches. Manual RTL design (VHDL/Verilog) offers maximum

microarchitectural control but is slow to develop, difficult to debug, and hard to scale across many models and design variants. High-Level Synthesis (HLS) reduces engineering effort by allowing C/C++-level descriptions but still demands significant optimization expertise to reach efficient implementations and is not inherently designed for rapid cross-model exploration [22]. Vendor deployment stacks such as Vitis AI provide optimized accelerators and runtime ecosystems, but they are constrained by platform support and workflow complexity: supported devices are limited to specific AMD/Xilinx families (and notably do not include Zynq-7000 boards such as ZedBoard), while deployment often requires a multi-stage toolchain with strict version and runtime alignment that can be cumbersome for research iteration [10], [23]. In contrast, automated CNN-to-FPGA toolflows such as fpgaConvNet and FINN aim to raise the level of abstraction by mapping networks to hardware through automated design-space exploration (DSE) and parameterized architectures [1], [20]. For this thesis, fpgaConvNet is particularly relevant because it supports multi-objective DSE, partitioning for feasibility, and targeting different FPGA devices through platform description files [1].

Finally, interoperability standards such as ONNX are increasingly important in modern deployment pipelines, enabling models trained in frameworks like PyTorch to be exported, validated in software (e.g., ONNX Runtime), and then fed into downstream hardware toolflows [4], [11]–[13]. In FPGA-centric pipelines, this ONNX-based "software reference → hardware mapping" loop is crucial for correctness validation and repeatable experimentation.

Research Gap. Despite strong progress in FPGA CNN acceleration research, two practical gaps remain highly relevant for student-scale and research workflows:

1. **Reproducible end-to-end "model-to-board" execution remains under-documented.** Toolflow papers often demonstrate impressive results, but reproducing an end-to-end pipeline (PyTorch → ONNX export → ONNX Runtime reference → fpgaConvNet parse/DSE/partition → Vivado build → host transfer → FPGA inference → logit comparison) still involves many fragile integration steps that are easy to get wrong and rarely described in full detail [1], [4], [11].

2. **Cross-model and cross-board exploration is limited by real hardware availability and platform coverage.** Many evaluations focus on one target board or a narrow set of devices, while practical projects frequently face the opposite reality: limited access to capable FPGA hardware. Even when a higher-resource board is available (e.g., KCU105), fully implementing and validating every model on real hardware can be disproportionately time-consuming relative to the insights gained—especially when tool-based feasibility and performance trends already reveal the dominant constraints.

3. **Board support and portability are bottlenecks for comparative research.** In practice, the ability to target additional FPGA platforms (even through consistent platform descriptions) directly affects how broadly results can be reproduced and extended. Expanding the set of compatible boards is therefore not just a convenience feature; it enables future model and design-space exploration when physical access to "ideal" hardware is restricted.

How this thesis addresses the gap. Based on these observations, this thesis focuses on a practical, ONNX-cantered fpgaConvNet workflow that is both reproducible and extensible. ZedBoard is used for embedded on-board validation, while a KCU105-targeted configuration is used to evaluate and compare multiple CNN families under a higher resource budget, enabling clearer feasibility boundaries and performance trends across models [1], [6]. In addition, to support future extension to more platforms and facilitate broader experimentation, the fpgaConvNet platform library is expanded with additional board/device descriptions, enabling consistent cross-board exploration

beyond the upstream set. This combination directly supports the thesis goal of implementing **more models** and enabling deployment exploration across **more FPGA targets**, while keeping the workflow grounded in what can be realistically executed and validated within the constraints of academic hardware access.

3. Methodology and Thesis Contributions

This chapter describes the practical methodology followed in this thesis and clarifies what was implemented beyond the baseline fpgaConvNet tutorial flow. The core objective is to build a reproducible end-to-end pipeline that starts from a trained CNN (PyTorch), exports it to ONNX, validates correctness in software (ONNX Runtime), maps the model through fpgaConvNet (parsing, design-space exploration, partitioning), generates an implementation through Vivado, and finally executes the design in a host–FPGA loop where FPGA logits are compared against the software reference [1], [11]–[13].

A key design decision in this work is to use **two complementary FPGA targets**: **ZedBoard** for embedded on-board validation (emulated/measured results and correctness checks) and **KCU105** for broader cross-model exploration via fpgaConvNet's device targeting (resource/performance estimation across many CNNs). Using both targets strengthens the study: ZedBoard provides "works on real hardware" evidence on a constrained embedded platform, while KCU105 provides a clearer view of scaling trends and feasibility boundaries under a larger resource budget.

3.1. End-to-End Workflow Overview

Figure 3.1 summarizes the full pipeline used throughout Chapters 3–5. The workflow is structured into four phases that align with how models are prepared, mapped, implemented, and validated:

1. **Preparation & Reference (Software)**: A CNN model is exported from PyTorch to ONNX and validated using ONNX Runtime to produce CPU reference logits and predictions [11]–[13].
2. **fpgaConvNet Mapping**: The ONNX model is provided to fpgaConvNet for parsing and mapping, where design-space exploration and partitioning are used to find feasible design points under FPGA constraints [1].
3. **Implementation (Vivado)**: The selected design is integrated into a Vivado project for synthesis, place-and-route, and bitstream generation [14].
4. **Execution & Validation (Host + FPGA)**: Inputs are transferred from host to FPGA, inference is executed, FPGA logits are read back, and correctness is validated by comparing against the ONNX Runtime reference (logits and/or predicted class) [11].

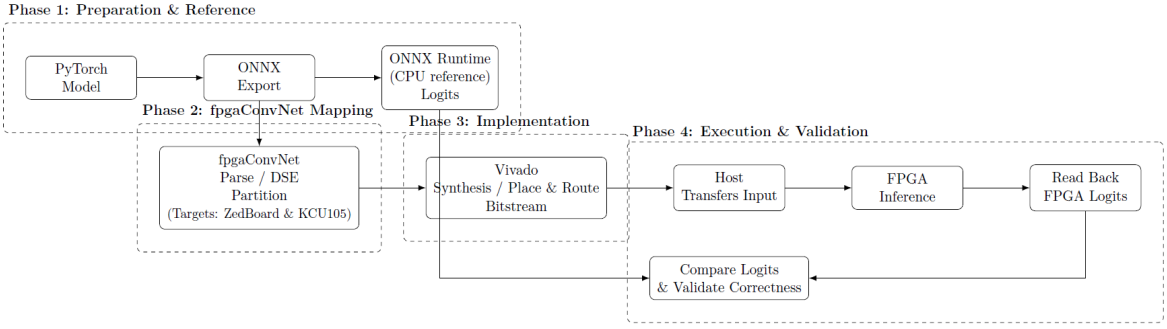


Figure 3.1: Workflow Overview

3.2. Deployment Targets and Practical Constraints (ZedBoard vs KCU105)

In practice, the available physical hardware for this thesis is limited. The boards available in-lab include **ZedBoard**, **KV260**, and **ZC702**, all of which have relatively limited FPGA resources for running larger CNNs. The only significantly stronger board available is **KCU105**, but creating a complete end-to-end Vivado implementation on KCU105 was not pursued because the required platform-specific Vivado integration and host/SDK adaptation was disproportionately complex for the purpose of this thesis (especially when the simulation-driven conclusions were already consistent with expectations).

Instead, this thesis adopts a **two-role strategy**:

- **ZedBoard (embedded validation):** Used for end-to-end execution, correctness validation, and measured/emulated timing on a real embedded Zynq-7000 platform. This provides concrete evidence that the toolflow can produce a working on-board deployment and that the host–FPGA loop (input transfer → inference → output readback) is functioning correctly.
- **KCU105 (cross-model exploration):** Used as a *target configuration* within fpgaConvNet (via device description files) to estimate resource utilization, partition counts, and performance trends across many models. These results support broader comparative evaluation (Chapter 5) without requiring full physical implementation on KCU105.

This separation also avoids conflating **tool-estimated** KCU105 results with **measured/emulated** ZedBoard results

3.3. Key Contributions

3.3.1. Reproducible Notebook-Based Pipeline (Key Contribution)

A major practical contribution of this thesis is improving reproducibility and usability of the fpgaConvNet workflow for **multiple models**, rather than only the default tutorial example. The official fpgaConvNet tutorials provide a working baseline, but the end-to-end process is split across multiple tutorial steps and is primarily demonstrated with a small set of models (e.g., a tutorial VGG-style network). To support systematic experimentation, the workflow was consolidated into reusable notebooks and scripts:

- **8_All_In_One_ONNX (created/extended in this thesis):** A unified notebook that combines the tutorial stages (model entry → parsing → exploration → partitioning → build steps) into a single reproducible pipeline, enabling faster iteration when switching models and targets.
- **9_End_To_End (created/extended in this thesis):** A simplified "raw ONNX load" path that focuses on quickly loading a model artifact and extracting performance-related signals, useful for debugging and collecting metrics without re-running unnecessary steps.
- **PY_To_ONNX (created in this thesis):** A helper conversion workflow used to export additional CIFAR-trained PyTorch checkpoints into ONNX, enabling broader model coverage than the default fpgaConvNet examples [12], [13].

These artifacts reduce the manual overhead required to repeat experiments and provide a clearer bridge between "tutorial baseline" and "research evaluation across models".

3.3.2. Extending fpgaConvNet Platform Support

fpgaConvNet supports multiple FPGA targets through board/platform description files (e.g., .toml specifications describing LUT/FF/BRAM/DSP budgets and other constraints) [1]. Since the thesis evaluation relies on comparing feasibility and scaling trends across targets, expanding the set of supported boards is a direct contribution.

The fpgaConvNet repository already supported a range of platforms, including:

AVNET Eval, CertusPro-NX Eval, CorAZ7, ECP5 Dev Kit, KV260, U50, U200, U250, VC707, VC709, VCU1525, ZC706, ZCU102, ZCU104, ZedBoard, ZU3EG, ZU7EV, Zybo.

As part of this thesis, the following additional platforms were added to expand the evaluation "arsenal":

AV Arty Z7-20, KC705, KCU105, KCU116, KCU1500, SP701, VCU108, VCU110, VCU118, ZC702, ZCU106, ZCU111, ZCU1275, ZCU1285.

This extension improves the generality of the experimental framework: even when physical boards are not available, fpgaConvNet can still be used to estimate feasibility and trends across a wider set of devices, supporting more meaningful comparative evaluation in Chapter 5 [1].

3.3.3. Expanding Model Coverage Beyond the Default Tutorial

The default fpgaConvNet tutorial flow is useful for validation, but it does not provide broad out-of-the-box support for many modern CNN architectures. This thesis therefore also focuses on expanding and testing model coverage using CIFAR-aligned pretrained networks [7], [8]. Across the evaluation work, the following outcomes were observed:

Works (successfully mapped / evaluated in the fpgaConvNet workflow):

- **MobileNetV2:** width multipliers ×0.5, ×0.75, ×1.0, ×1.4
- **ResNet:** ResNet20, ResNet32, ResNet44, ResNet56
- **VGG with BatchNorm:** VGG11-BN, VGG13-BN, VGG16-BN, VGG19-BN

Does not work (toolflow/parser limitations encountered):

- **RepVGG variants (a0, a1, a2):** not supported in the current flow due to unsupported patterns in the exported graph (notably around normalization/re-parameterized blocks).
- **ShuffleNetV2 (×0.5, ×1.0, ×1.5, ×2.0):** severe ONNX parsing issues leading to node-level failures.

Importantly, achieving this broader model coverage required non-trivial modifications to the downloaded fpgaConvNet codebase and a dedicated ONNX conversion workflow, meaning that a fresh upstream fpgaConvNet installation would not necessarily reproduce these results without applying the same changes. These limitations and model-support outcomes are reflected in the evaluation results and discussed further in Chapter 5.

3.4. Case Study A: Tutorial Baseline on ZedBoard

To establish a reliable baseline for correctness and execution, the work begins from the official fpgaConvNet tutorial flow on **ZedBoard** [1]. The purpose of this baseline is not to claim novelty in the tutorial itself, but to provide:

- A verified end-to-end directory/build structure,

- A working reference for Vivado integration,
- A known-good host-to-FPGA execution path, and
- A template for later ONNX-based deployments and multi-model experimentation.

In the baseline setup, the tutorial model (VGG-style network) is mapped and implemented following the standard fpgaConvNet procedure, producing a design that can be executed in an on-board validation loop. This baseline is essential because it de-risks later stages: once a working end-to-end system exists, subsequent experiments can focus on changing the **model** (ONNX input) and **mapping parameters**, rather than debugging the entire hardware/software integration from scratch.

3.5. Summary

This chapter presented the practical methodology used throughout the thesis and highlighted the main contributions that enable broader, reproducible evaluation:

- An end-to-end pipeline from PyTorch/ONNX software reference to fpgaConvNet mapping and on-board validation,
- A clear two-target strategy (ZedBoard for real embedded validation, KCU105 targeting for comparative exploration),
- Consolidation of tutorial steps into reusable notebooks for multi-model experimentation,
- Expansion of fpgaConvNet platform support by adding additional FPGA board descriptions, and
- Expanded CNN model coverage (with documented limitations for unsupported architectures).

The next chapter (Chapter 4) builds directly on this foundation and describes the ONNX-based deployment path and the host–FPGA validation methodology in detail, preparing the ground for the quantitative evaluation in Chapter 5.

4. Case Studies

This chapter presents the case studies that were implemented and evaluated in this thesis. Unlike the tutorial baseline (moved to Chapter 3 as background work), the focus here is on the **original, thesis-specific** end-to-end pipeline used to deploy ONNX models, validate correctness against a software reference, and collect the metrics used later in the evaluation chapter. The chapter also clarifies the practical role of each FPGA target used in this work: **ZedBoard** is used for on-board validation and measured execution, while **KCU105** is used as a higher-resource target configuration for fpgaConvNet estimation-based cross-model evaluation [1].

4.1. Experimental Context and Targets

The experiments in this thesis were developed under practical hardware and toolchain constraints typical of academic FPGA research. Although several FPGA boards were available (e.g., ZedBoard, KV260, ZC702), their resources are limited for evaluating larger CNN families end-to-

end. A more capable board (KCU105) was also accessible; however, implementing a complete Vivado deployment for every explored model and configuration would have required substantial additional platform-specific integration effort. Since the comparative trends and feasibility boundaries across models can already be revealed through fpgaConvNet's analytical estimation framework, this thesis adopts a split strategy:

- **ZedBoard (Zynq-7000):** used for end-to-end deployment, on-board execution, correctness validation, and measured/emulated metrics in a real hardware loop.
- **KCU105 (Kintex UltraScale):** used as a **target configuration** for fpgaConvNet to estimate feasibility, resource utilization, partition count, and performance trends across multiple models under a larger resource budget [1].

This separation avoids conflating measured results with tool-estimated results and enables a broader model study without requiring full board bring-up for each design point.

4.2. ONNX-Based Deployment and Validation on ZedBoard

This case study implements the primary pipeline used to deploy CNN models in an ONNX-based flow and validate output correctness on a real embedded FPGA platform. The workflow follows four steps: (i) ONNX export, (ii) CPU reference inference, (iii) hardware mapping/build, and (iv) on-board execution and output comparison.

4.2.1. Model preparation and ONNX export

Models were prepared in PyTorch and exported to the **Open Neural Network Exchange (ONNX)** format using the PyTorch ONNX exporter [12], [13]. ONNX provides a stable graph representation that supports reproducible deployment across backends and enables models to be validated in software and later mapped to hardware toolflows [11]–[13]. For systematic evaluation across architectures, pretrained CIFAR-based models were used as the starting point, enabling consistent comparisons while reducing training overhead [7], [8].

In practice, ONNX export is not only a file conversion step; it also determines what operators and graph patterns appear in the deployed network. Since fpgaConvNet relies on parsing the ONNX graph, compatibility depends on whether the exported graph contains supported operators and forms. For this reason, ONNX export and graph inspection are treated as part of the deployment methodology, especially when switching across CNN families.

4.2.2. Software reference inference with ONNX Runtime

Before running on the FPGA, **ONNX Runtime** was used to execute the exported ONNX model on CPU and produce a reference output [11]. For each test input, ONNX Runtime produces:

- Reference logits (floating point), and
- A reference predicted class (argmax over logits).

This reference output serves two purposes. First, it validates that the exported ONNX model matches the expected behaviour of the source model. Second, it provides a ground-truth comparison target for on-board correctness checks after FPGA execution. Using ONNX Runtime as a reference reduces ambiguity in debugging: if a mismatch occurs, it can be traced to fixed-point representation, interface/packing, or toolflow mapping rather than to training or framework inconsistencies [11]–[13].

4.2.3. fpgaConvNet mapping and Vivado build

After software validation, the ONNX model is provided to **fpgaConvNet** for mapping to an FPGA architecture [1]. fpgaConvNet uses a dataflow formulation and performs design-space exploration to select parallelism and buffering parameters under platform constraints. When a full network does not fit within the target resources, fpgaConvNet can partition the network into multiple feasible subgraphs ("partitions"), which can then be executed sequentially to maintain functional correctness under limited resources [1]. This behaviour is fundamental to the evaluation chapter, where partition count and feasibility are used as indicators of scalability across models.

For ZedBoard, the resulting design is integrated into the Vivado toolchain for synthesis, place-and-route, and bitstream generation. This thesis uses **Vivado 2019.1** for implementation to match the established fpgaConvNet tutorial environment and ensure toolflow compatibility [14].

4.3. Host–FPGA Interface and Output Verification (ZedBoard)

The ZedBoard case study is executed in a hardware-in-the-loop setup using a host application that manages input transfer, execution triggering, and output collection. Conceptually, execution follows:

1. **Input preparation:** the host preprocesses an input sample into the numeric format expected by the accelerator.
2. **Transfer to FPGA:** the processed input is sent to the board (in this thesis setup, via a UART-based interface in the tutorial-style design).
3. **FPGA inference:** the accelerator executes inference and produces output logits.
4. **Output readback:** logits are read back on the host.
5. **Correctness check:** FPGA outputs are compared against ONNX Runtime reference outputs.

A critical detail is that FPGA accelerators often operate with **fixed-point arithmetic** for efficiency. Therefore, the output logits from the FPGA are commonly represented as signed integers (e.g., **int16**). To compare fairly against floating-point CPU logits, the host decodes the FPGA outputs using the expected signed representation and applies the deployment scale factor used by the design (e.g., dividing by a constant such as 256 in the demonstrated example). Correctness validation is then performed by checking that:

- The predicted class (argmax) matches the ONNX Runtime reference, and
- The relative ordering/magnitude pattern of logits is consistent within expected quantization effects [1], [11].

This comparison step is essential: it demonstrates that the end-to-end pipeline is functionally correct even when numerical differences are expected due to reduced precision.

4.4. Custom End-to-End Debug Configuration (ZedBoard)

Beyond the main deployment pipeline, a second case-study configuration was developed to enable faster debugging and metric extraction. The purpose is to isolate the core operations needed for end-to-end execution while removing unnecessary dependencies:

- Loading a model artifact (or simplified deployment entry),
- Generating or reusing the FPGA design,

- Sending an input and receiving outputs, and
- Verifying output decoding.

This configuration is useful when investigating issues such as parsing failures, mismatched preprocessing, incorrect packing/unpacking, or inconsistent output scaling. It complements the full pipeline by providing a minimal path to reproduce and diagnose errors.

4.5. Models and Datasets Used in the Case Studies

The experiments in this chapter are designed to support the broader evaluation in Chapter 5. For reproducibility and consistency across architectures:

- **CIFAR-10/100** provide the dataset context for the evaluated CNN families and are widely used for small-image classification benchmarking [7].
- **Pretrained CIFAR models** were obtained from a public repository to enable consistent comparisons across architectures without requiring re-training [8].
- Where tutorial-style validation examples are referenced, MNIST-style digit classification is considered a standard lightweight benchmark for embedded inference pipelines [21].

The CNN architectures explored in the wider evaluation include VGG-BN, ResNet, and MobileNetV2 variants, selected to provide diversity in compute structure and hardware mapping complexity [15]–[18].

4.6. Reproducibility and Artifacts

To make the workflow repeatable and suitable for multi-model experimentation, the thesis artifacts include:

- The fpgaConvNet tutorial structure as a baseline reference for build and deployment steps [9],
- ONNX export and ONNX Runtime inference as the software reference path for correctness validation [11]–[13], and
- Consolidated notebooks/scripts used to automate model conversion, deployment orchestration, and output verification.

In addition, fpgaConvNet platform support was extended with additional board descriptions to enable broader cross-target evaluation, even when physical access to those boards is limited (details in Chapter 3).

4.7. Summary

This chapter presented the thesis-specific case studies used to implement and validate the end-to-end workflow. The ZedBoard case study demonstrates on-board execution and correctness verification through CPU–FPGA logits comparison, establishing a functional deployment pipeline. A custom debug configuration supports rapid diagnosis and repeatable measurement. Together, these case studies provide the implementation foundation required for the quantitative evaluation in Chapter 5, where measured ZedBoard results and KCU105-targeted fpgaConvNet estimates are analysed across multiple CNN architectures.

5. Evaluation

This chapter presents the experimental evaluation of the implemented fpgaConvNet-based workflow. Two FPGA targets are used with clearly separated roles: (i) ZedBoard (Zynq-7000), used for implemented on-board validation and measured execution results, and (ii) All KCU105 results in this chapter are **fpgaConvNet tool-estimated** values derived from its analytical models and the selected platform description. These results indicate comparative feasibility and scaling trends across models, but they are **not post-place-and-route timings** and do not include board-level I/O overheads. Therefore, KCU105 results are interpreted as **design-space exploration indicators** rather than guaranteed achieved performance on silicon. Models were exported from PyTorch to ONNX for a consistent representation across software validation and FPGA deployment [12], [13], and ONNX Runtime was used as the CPU reference for correctness checks [11].

5.1. Evaluation Objectives and Metrics

The evaluation has three main objectives:

1. **Validate functional execution on real hardware (ZedBoard)** by confirming that the end-to-end host–FPGA pipeline runs correctly and produces sensible outputs.
2. **Quantify performance** in terms of **latency** and **throughput** for the implemented prototype (ZedBoard).
3. **Compare model scalability and feasibility (KCU105)** across a set of CNN architectures, by analysing:
    - **Final partition count** (how many fpgaConvNet partitions were required to map the network),
    - **FPGA resource utilization** (LUT, FF, BRAM, DSP),
    - **Estimated latency and throughput** for each candidate model.

In this chapter, “latency” refers to the per-inference execution time, and “throughput” refers to the achievable inferences per second under batch-1 settings (unless otherwise stated). For multi-model comparisons on the KCU105, the reported values are **tool-estimated** results produced by fpgaConvNet’s modelling/estimation flow [1], rather than directly measured on hardware.

5.2. ZedBoard: Implemented Prototype Measurements

The ZedBoard platform was used to validate the complete pipeline (host preprocessing → FPGA inference → UART readback of outputs). The measured metrics focus on the compute-side timing as reported by the FPGA design during execution.

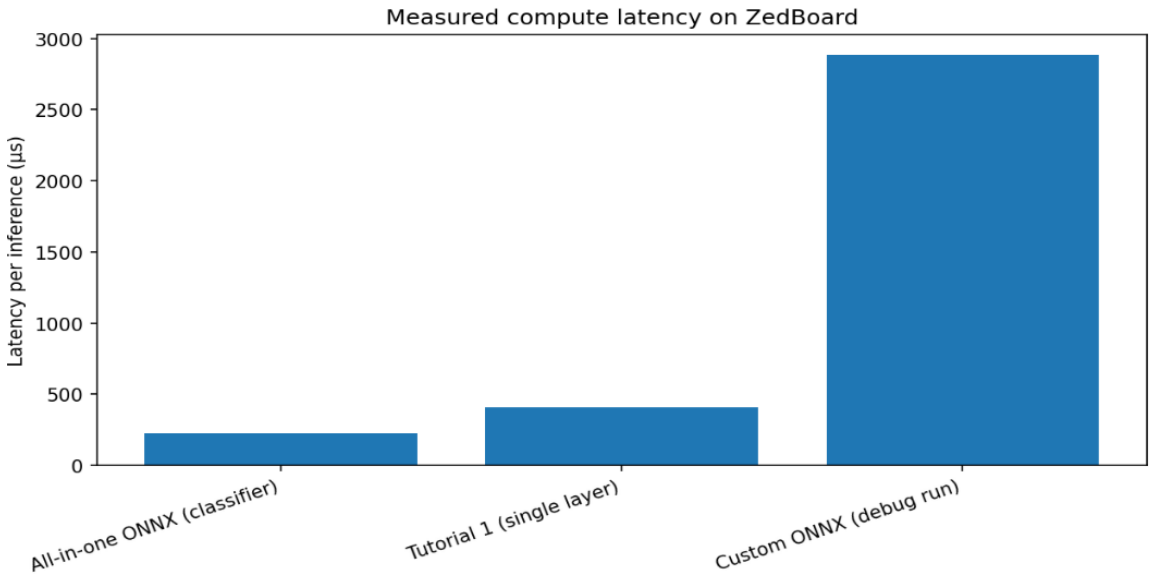


Figure 5.1: ZedBoard measured compute latency

Figure 5.1 shows the measured compute latency for three representative configurations (tutorial baseline, integrated ONNX design, and a custom end-to-end debug configuration). The integrated "all-in-one" ONNX pipeline achieves the lowest compute latency (≈0.22 ms), while the custom debug configuration is slower due to additional overheads and debugging-oriented settings.

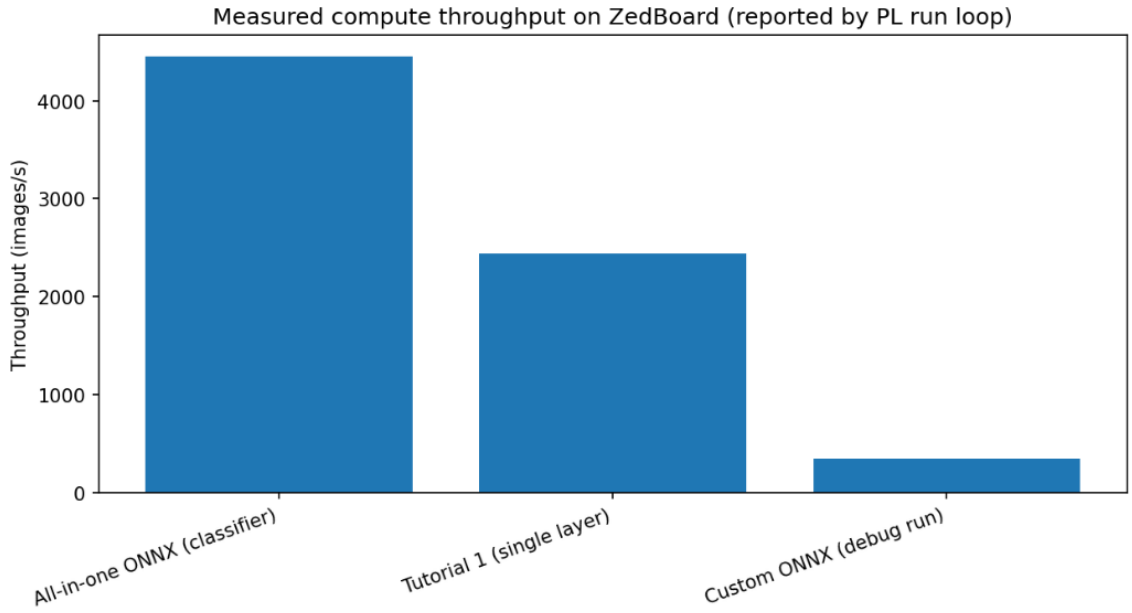


Figure 5.2: ZedBoard measured compute throughput

Figure 5.2 reports the corresponding throughput. The "all-in-one" ONNX configuration achieves the highest throughput (≈4.45k inferences/s), while the other configurations achieve lower throughput due to reduced parallelism and/or additional overheads.

These results confirm that the ZedBoard implementation is capable of executing the mapped network at low compute latency, demonstrating the feasibility of the fpgaConvNet-generated accelerator pipeline on an embedded FPGA target [1].

5.3. ZedBoard: Output Verification (CPU vs FPGA)

To validate correctness at the system level, FPGA outputs were compared against a CPU reference inference path. This verification is important because FPGA deployments typically use fixed-point arithmetic and quantization, meaning raw output values may differ in scale from floating-point CPU inference even when the predicted class matches.

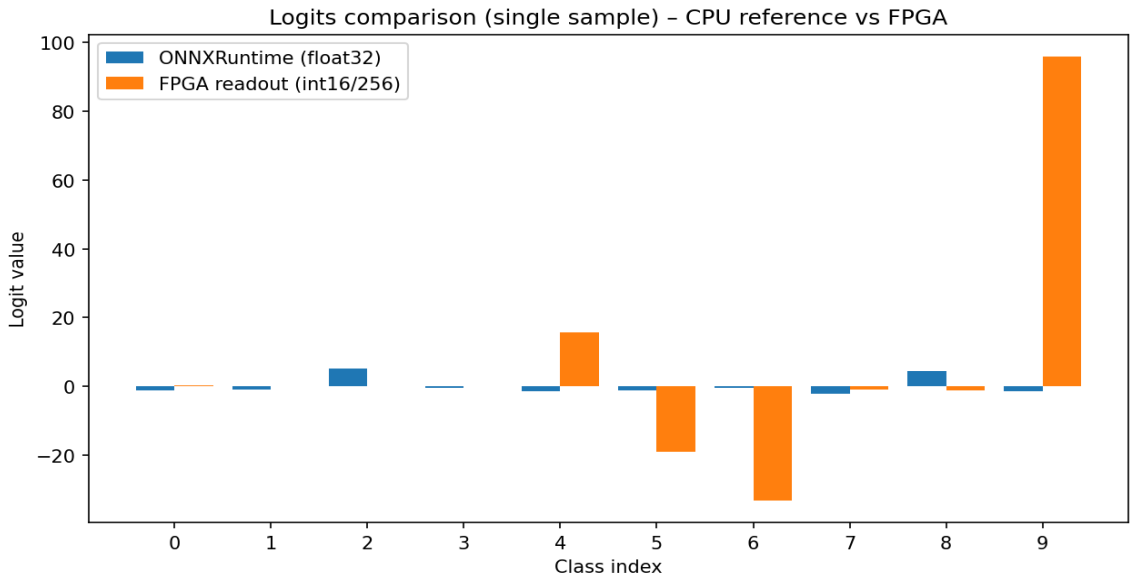


Figure 5.3: CPU vs FPGA logits comparison

Figure 5.3 illustrates a representative comparison of output logits between CPU and FPGA. In our host-side decoding, FPGA outputs are interpreted as **signed int16 logits** and scaled to floating-point for plotting using the same factor as the deployment format (e.g., **int16/256** in the shown example), before comparing against ONNX Runtime reference logits. This step is part of the standard "software–hardware agreement" process when models are exported (e.g., via ONNX) and mapped into a hardware-friendly fixed-point representation, which typically requires signed interpretation and an appropriate scaling factor for comparison [1], [4]. Overall, this comparison step provides confidence that the end-to-end data path (including UART transmission and output decoding) is functioning as intended.

5.4. KCU105: Multi-Model Evaluation Setup

Beyond the ZedBoard prototype, a broader model-level evaluation was conducted using the **KCU105** as a target FPGA device for fpgaConvNet's estimation flow. The goal was to understand how different CNN architectures scale in terms of:

- required partitioning,
- FPGA resource utilization,
- estimated latency/throughput trade-offs,
- feasibility under the target device constraints.

The evaluated model set includes **ResNet** variants (ResNet20/32/44/56), **VGG with batch normalization** (VGG11/13/16/19 BN), and **MobileNetV2** with multiple width multipliers (×0.5, ×0.75, ×1.0, ×1.4). This type of comparative evaluation is aligned with the broader motivation of exploring architecture–hardware trade-offs, which is also central in dynamic/adaptive CNN deployment research [4], [5].

5.5. KCU105: Partitioning Behaviour

fpgaConvNet may split a network into multiple partitions to satisfy on-chip resource constraints and mapping limitations [1]. A higher partition count usually indicates either a deeper/more complex network, stricter per-partition resource limits, or both.

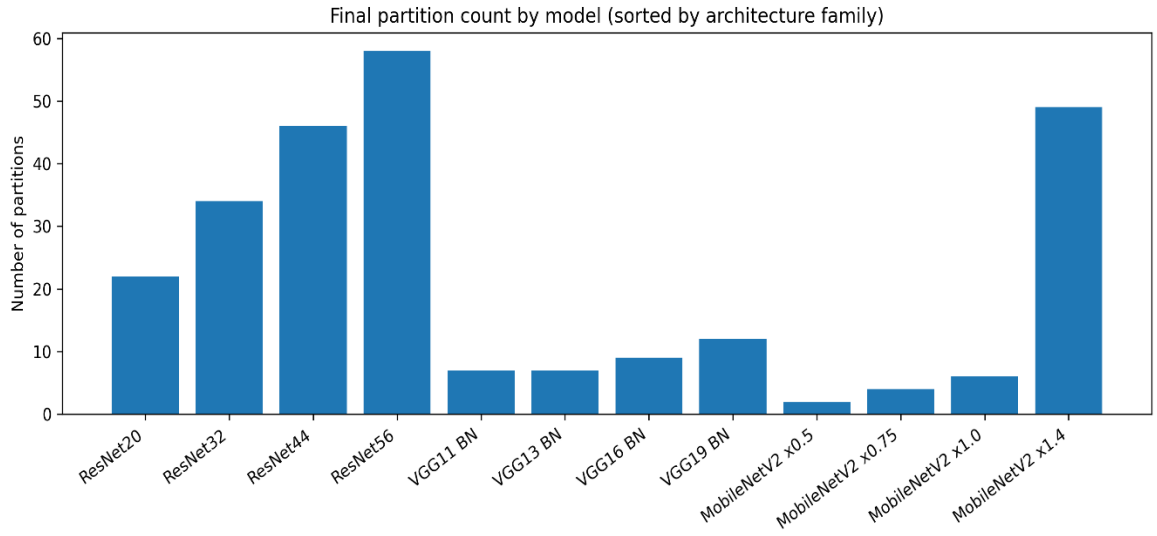


Figure 5.4: Final partition count by model

Figure 5.4 shows the final partition counts for each evaluated model. The ResNet family generally exhibits increasing partition counts as depth increases (e.g., ResNet20 → ResNet56), while the largest MobileNetV2 configuration (×1.4) also produces a high partition count, reflecting its increased compute/memory pressure.

5.6. KCU105: Estimated Resource Utilization and Feasibility

Resource utilization determines whether a mapped design can fit on the FPGA. The primary resources considered are:

- **LUTs** (logic),
- **FFs** (registers),
- **BRAM** (on-chip memory),
- **DSP blocks** (multiply–accumulate support).

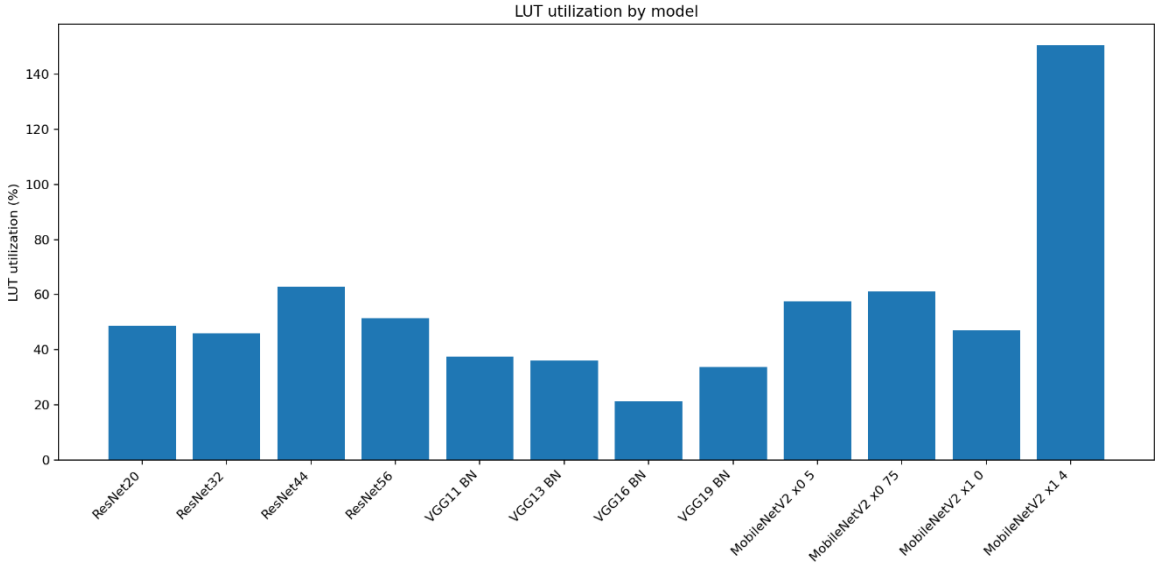


Figure 5.5: LUT utilization (%) by model

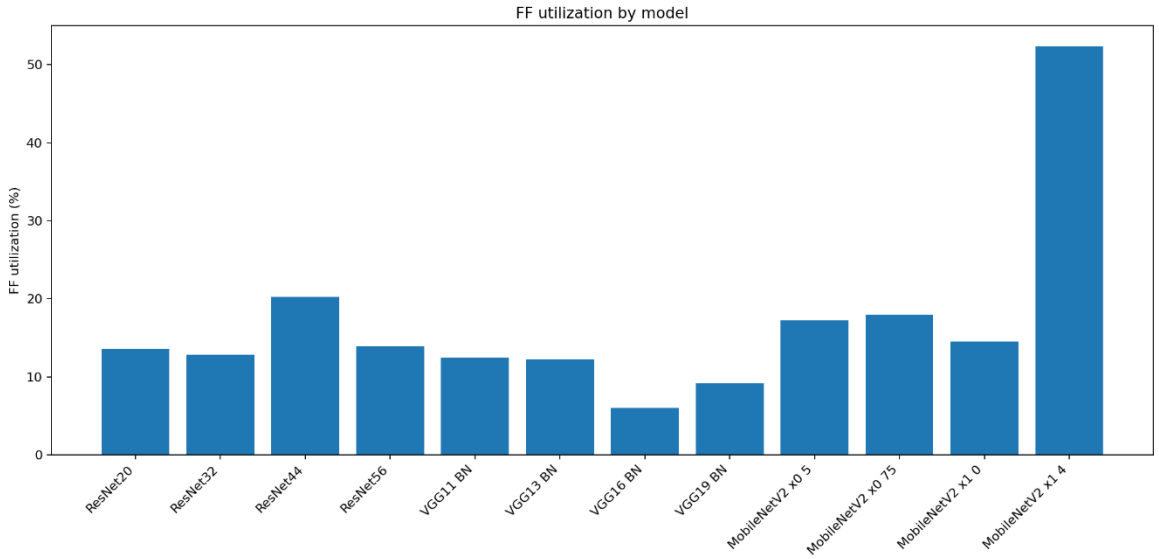


Figure 5.6: FF utilization (%) by model

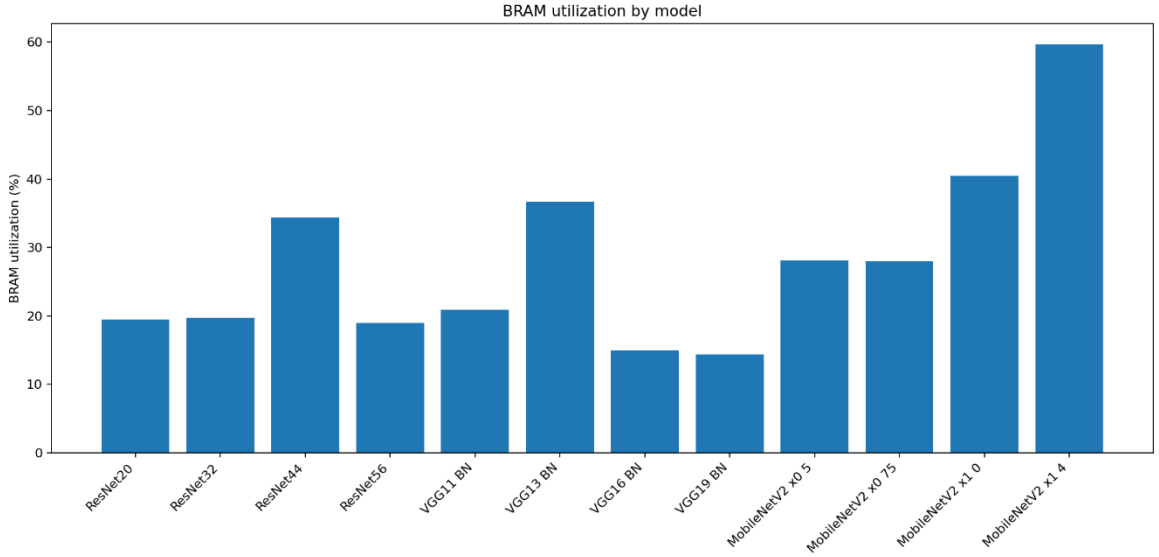


Figure 5.7: BRAM utilization (%) by model

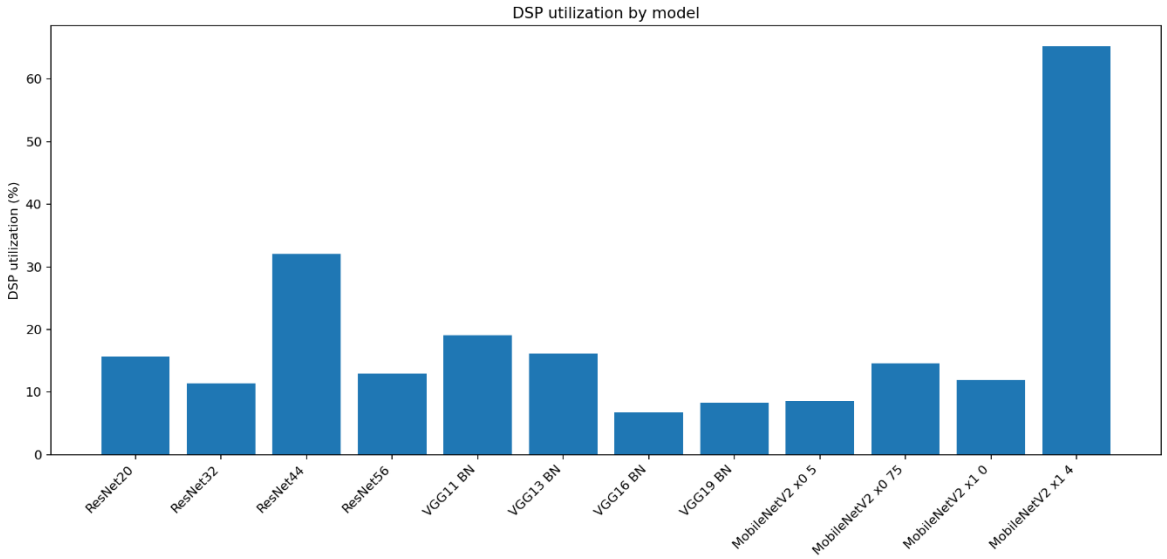


Figure 5.8: DSP utilization (%) by model

From these figures, the most important observation is that **MobileNetV2 ×1.4 exceeds the available LUT resources (≈150%)**, meaning it is **infeasible** on the target device under the explored configuration. In contrast, all other evaluated models remain below 100% utilization across the reported resources, and are therefore feasible candidates for deployment (subject to implementation details and timing closure).

This highlights a practical constraint often seen in FPGA CNN mapping: **LUT availability can become the dominant limiting factor**, especially for wider networks. This is consistent with the general challenge of mapping high-capacity CNNs under fixed FPGA resource budgets [1], [6].

5.7. KCU105: Estimated Latency and Throughput Trends

After feasibility is established, performance becomes the key differentiator. The estimated latency and throughput results provide insight into which architectures offer the best speed under the same FPGA target constraints.

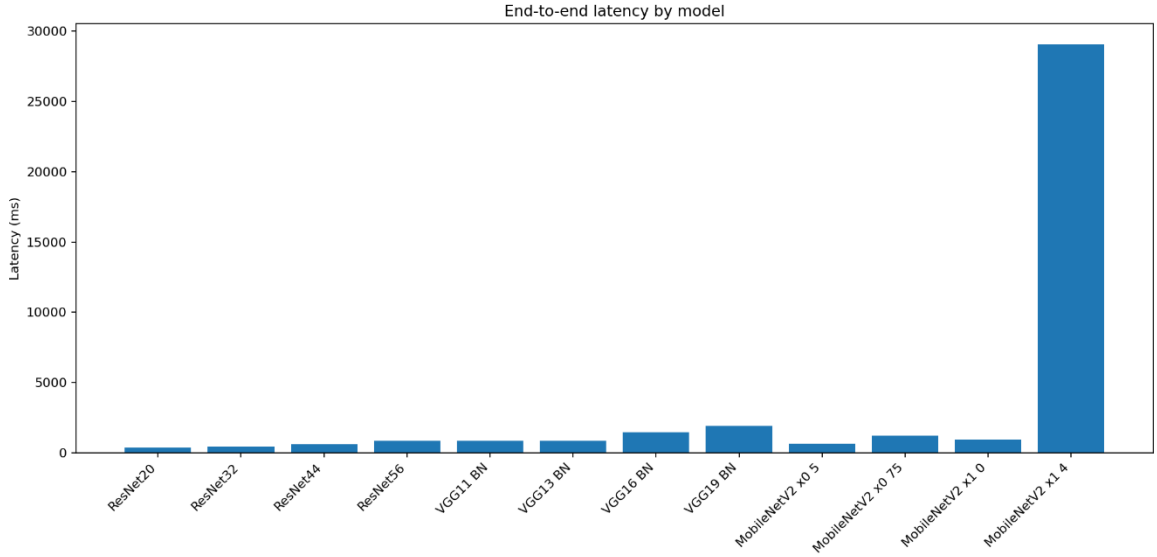


Figure 5.9: Latency by model

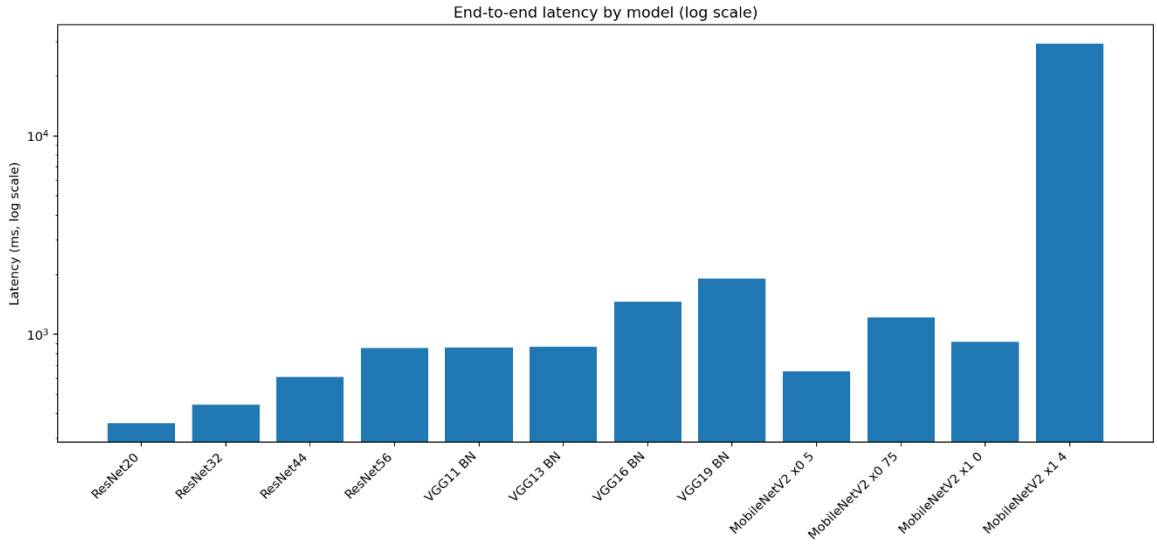


Figure 5.10: Latency by model (log scale)

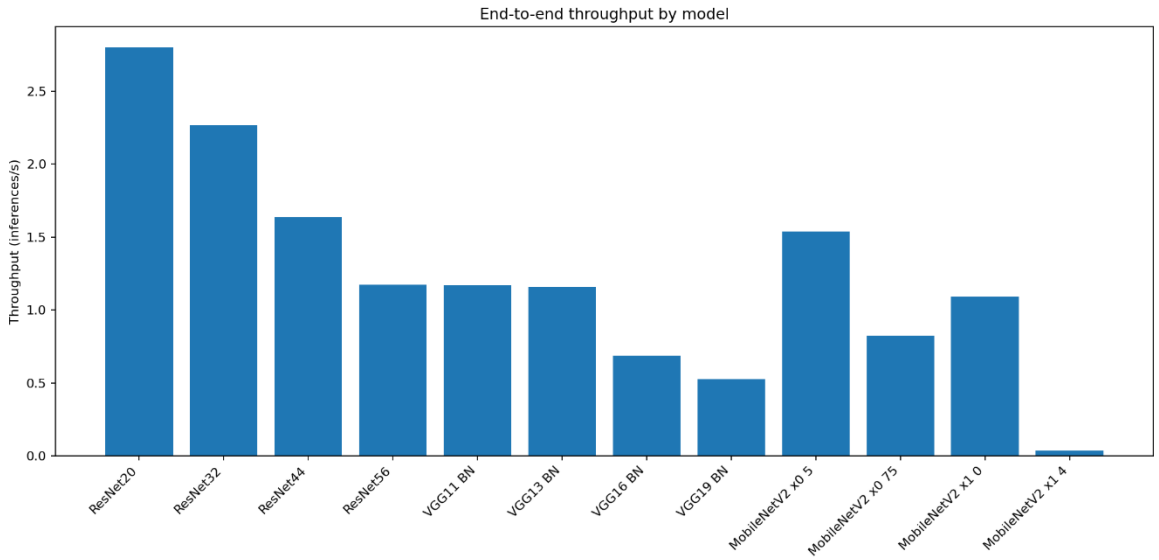


Figure 5.11: Throughput by model

The key trends are:

- **ResNet20** achieves the best estimated performance among the evaluated set, with the lowest latency and highest throughput.
- Increasing ResNet depth generally reduces throughput and increases mapping complexity (as also reflected by partition count).
- **VGG16/19 BN** exhibit lower throughput and higher latency compared to the ResNet variants, reflecting their heavier convolutional workload and memory behaviour.
- **MobileNetV2** shows strong performance at smaller widths (×0.5, ×1.0), but scaling to ×1.4 becomes infeasible due to LUT limits, illustrating the cost of increasing channel width on FPGA resources.

To better visualise the joint trade-off between resource usage and speed, LUT utilization can be plotted directly against throughput while using partition count as an additional indicator.

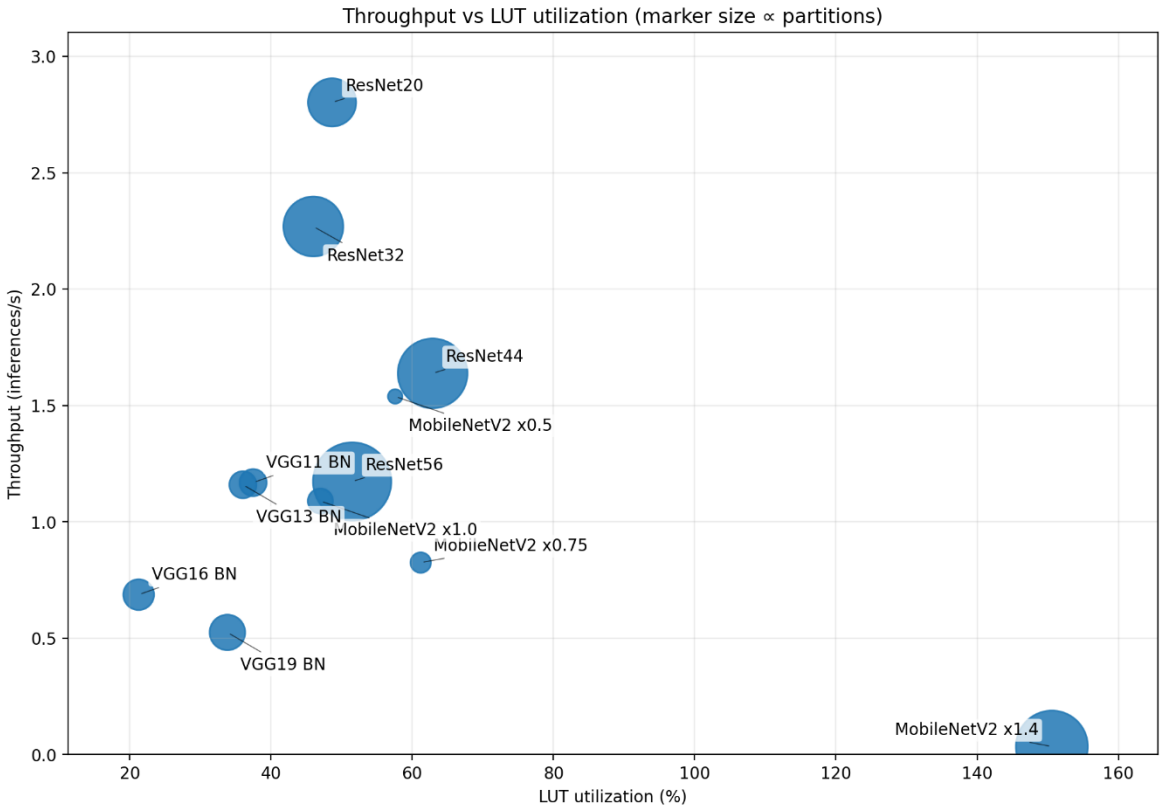


Figure 5.12: Throughput vs LUT utilization (marker size ∝ partitions)

Figure 5.12 shows that the highest-throughput designs appear at moderate LUT utilization, while pushing utilization beyond device capacity results in infeasibility. The partition-size marker also helps explain why certain models may deliver lower throughput despite acceptable resource usage: partitioning and architectural structure influence the final pipeline efficiency [1].

5.8. Summary of Findings

Overall, Chapter 5 demonstrates:

- **ZedBoard** successfully supports the implemented fpgaConvNet end-to-end prototype and achieves low compute latency with high throughput for the validated configuration.
- **KCU105** enables broader comparative evaluation across multiple CNN architectures, revealing clear feasibility boundaries and performance trends across model families.
- **LUT utilization is the primary limiting resource** for the evaluated set, with MobileNetV2 ×1.4 exceeding device limits.
- The results reinforce the practical need for architecture selection and (in broader contexts) dynamic/adaptive strategies to meet hardware constraints while delivering acceptable performance [4], [5].

6. Conclusions

6.1. Summary of Thesis Outcomes

This thesis investigated the practical acceleration of convolutional neural network (CNN) inference on field-programmable gate arrays (FPGAs), with an emphasis on creating a reproducible end-to-end workflow and systematically evaluating performance and resource trade-offs across different CNN architectures and FPGA device configurations. The work focused on fpgaConvNet due to its streaming, dataflow-oriented approach and its built-in capability for design-space exploration (DSE) across multiple hardware targets [1]. While the implementations explored in this thesis are static, the methodology and results were considered in the broader context of dynamic and adaptive CNN execution, motivated by recent work on dynamic neural networks and altitude-dependent CNN parameter selection for UAV tasks [2], [3], as well as ONNX-based adaptive FPGA inference flows [4].

6.1.1. Thesis Contributions

The main contributions of this thesis can be summarised as follows:

C1. Practical comparison of alternative FPGA deployment approaches.

Early in the project, multiple approaches were explored, including vendor-oriented toolflows and HLS-based paths, before selecting fpgaConvNet as the primary methodology. This exploration highlighted real-world constraints such as platform support, setup friction, and portability trade-offs that often determine whether a workflow is feasible in practice, beyond purely theoretical performance considerations.

C2. End-to-end ONNX-to-FPGA workflow on an embedded FPGA platform.

A complete workflow was developed that starts from trained CNN models (based on CIFAR-style architectures and pretrained baselines [7], [8]), exports them to ONNX, and processes them using fpgaConvNet for mapping and accelerator generation [1]. The workflow was implemented and documented through custom Jupyter notebooks that consolidate and extend fpgaConvNet tutorial pipelines into a reproducible process that includes parsing, optimisation, high-level synthesis, and bitstream generation.

C3. Case-study evaluation across CNN models and FPGA targets (and platform-library extension).

This thesis includes an empirical evaluation of multiple CNN architectures under fpgaConvNet using two FPGA target configurations: **ZedBoard** (used for emulated / on-board validation) and **KCU105** (used to evaluate how different models scale on a higher-resource FPGA through tool-based estimates). In addition, as a practical contribution, the fpgaConvNet **platform library was extended** to expand supported targets beyond the upstream set by adding new platform/device descriptions for: **AV Arty Z7-20, KC705, KCU105, KCU116, KCU1500, SP701, VCU108, VCU110, VCU118, ZC702, ZCU106, ZCU111, ZCU1275, and ZCU1285**. This extension improves the portability and reproducibility of the workflow by enabling consistent cross-board exploration even when physical hardware access is limited. [1], [6].

C4. Positioning the results toward dynamic and adaptive FPGA inference.

Although the thesis focuses on static accelerators, the outcomes were interpreted in light of dynamic DNN research and adaptive inference motivations. In particular, the results provide baseline insights that (i) support future dynamic/altitude-aware deployment strategies (as in multi-altitude UAV parameter exploration) [2], (ii) align with the challenges and directions identified for dynamic DNN acceleration on FPGAs [3], and (iii) motivate future integration with ONNX-centric adaptive and reconfigurable inference pipelines [4].

6.1.2. Key Findings from ZedBoard On-Board Measurements

A central result of the thesis is that fpgaConvNet can be used to bridge the gap between high-level CNN models and real FPGA execution on an embedded development board. The ZedBoard case studies demonstrated that, once a working host-to-FPGA pipeline is established, the accelerator compute itself can achieve very low per-inference compute latency at the hardware kernel level. In the measured compute-only experiments, the best-performing configuration achieved approximately **224 µs** compute latency per inference (corresponding to roughly **4.45k inferences/s** at the accelerator level). Other experimental variants demonstrated higher compute latency (hundreds of microseconds to a few milliseconds), illustrating how design choices and pipeline configuration can significantly impact realised performance.

In addition to performance, functional correctness was validated by comparing FPGA-produced logits against a software reference (ONNX runtime inference). The experiments showed that the FPGA output preserved the correct predicted class, while small differences in raw logit magnitudes were expected due to fixed-point representation, scaling, and quantisation effects that arise when mapping neural networks to FPGA-friendly arithmetic. These findings support the thesis argument that fpgaConvNet provides a viable, practically usable pathway for deploying CNN inference on embedded FPGAs in a student-scale workflow [1].

6.1.3. Insights from KCU105-Based Multi-Model Evaluation and Design-Space Exploration

To go beyond a single on-board deployment, fpgaConvNet's ability to target different devices via platform description files was used to evaluate multiple CNN architectures under a higher-resource FPGA target. In particular, the **KCU105 device configuration** was used to compare CNN families including **ResNet (20/32/44/56)**, **VGG (11/13/16/19 with batch normalisation)**, and **MobileNetV2 variants (width multipliers 0.5, 0.75, 1.0, 1.4)**. This enabled systematic analysis of how architectural characteristics affect mapping feasibility, partitioning behaviour, and performance-resource trade-offs, consistent with fpgaConvNet's optimisation-driven mapping philosophy [1].

Several consistent trends emerged:

- **Model complexity vs. throughput/latency:** deeper or heavier models generally reduced throughput and increased latency, reflecting higher computational demand and increased pressure on FPGA resources.
- **Resource feasibility constraints:** the evaluation highlighted that some model/device combinations can become infeasible or highly inefficient, visible through extreme resource requirements and excessive partitioning. A key example was **MobileNetV2 ×1.4**, which exceeded available LUT resources in the evaluated configuration and exhibited a dramatic performance collapse, effectively acting as an "infeasible mapping" indicator in the results.

- **Partition count as a practical indicator:** the final partition counts differed substantially across models. In general, larger partition counts reflected increasing difficulty in fitting and scheduling the design within resource and bandwidth constraints and were associated with poorer end-to-end performance. This reinforces the practical importance of partitioning and mapping granularity when using streaming FPGA accelerators.

Overall, the multi-model evaluation confirmed that fpgaConvNet's DSE-style workflow is valuable not only for producing a working accelerator, but also for understanding feasibility boundaries and performance scaling across architectures and device resource budgets [1], [6]. These insights are especially relevant when the longer-term goal is to support adaptive or dynamic execution, since dynamic systems must often operate within strict feasibility margins and may require multiple valid "profiles" rather than a single monolithic mapping [2]–[4].

6.1.4. Limitations

This thesis has several limitations that should be acknowledged:

1. **Static accelerators only:** all implemented accelerators are static; no runtime switching, early-exit behaviour, channel gating, or partial reconfiguration was implemented. Dynamic DNN execution remains substantially more complex due to variable control flow, scheduling challenges, and toolchain limitations [3].
2. **Device coverage and measurement scope:** the ZedBoard was used for measured, on-board experiments, while broader multi-model evaluation relied on fpgaConvNet's target-based evaluation using the KCU105 device configuration. As a result, not all explored models were implemented and timed as fully deployed hardware bitstreams on physical boards.
3. **End-to-end overheads:** practical throughput in real deployments is strongly influenced by I/O, data movement, host preprocessing, and communication overheads. While compute-only measurements on the FPGA kernel can be extremely fast, end-to-end system throughput may be dominated by non-kernel components unless the host–FPGA interface and buffering are carefully engineered.
4. **Limited exploration of power and energy:** while FPGAs are often motivated by performance-per-watt benefits [1], [6], this thesis prioritised functional deployment and latency/throughput/resource evaluation. Detailed power measurement and energy-per-inference evaluation were not treated as a primary metric.
5. **Quantisation and accuracy trade-offs:** the work observed expected numerical differences between FPGA and software logits due to fixed-point arithmetic but did not conduct a full accuracy-vs-precision study across quantisation schemes and calibration methods.

6.1.5. Future Work

Future work can extend this thesis in several high-impact directions:

1. **Full hardware validation across multiple FPGA boards.**
   A natural next step is to implement and measure additional CNNs as deployed bitstreams on multiple physical FPGA boards, including higher-resource platforms where more complex models may be feasible. This would strengthen the empirical comparison between estimated DSE outcomes and real post-implementation timing results.
2. **Improved end-to-end system engineering.**
   To translate fast kernel execution into high practical throughput, future work should focus

on optimising the host–FPGA pipeline (data streaming, buffering, batching, DMA-capable interfaces where possible) and carefully separating compute latency from I/O and control overhead.

3. **Dynamic and altitude-aware CNN adaptation.**
   The strongest long-term motivation is to progress from static accelerators to **adaptive inference** in which model parameters (e.g., resolution, width, kernel size) are adjusted according to context such as UAV altitude. This aligns directly with dynamic CNN parameter exploration findings in multi-altitude UAV object detection, where different operating regimes favour different accuracy–compute trade-offs [2]. FPGA-based support for such adaptation requires mechanisms for fast switching between profiles, dynamic scheduling, or hardware reconfiguration.

4. **Runtime profile selection and coarse-grained reconfigurable designs.**
   ONNX-to-hardware approaches that support multiple execution profiles and runtime selection provide a promising blueprint [4]. Future work could combine fpgaConvNet-style mappings with a multi-profile runtime manager, enabling several pre-optimised accelerator variants to be selected at runtime based on accuracy constraints, energy availability, or mission requirements.

5. **Partial reconfiguration and dynamic DNN tool support.**
   Dynamic DNN acceleration on FPGAs remains an active research area with known challenges in tool support, control complexity, and reconfiguration overhead [3]. Future work could investigate partial reconfiguration to reduce switching cost between profiles, or explore architectures that keep multiple configurations resident at the cost of additional resources, as discussed in dynamic DNN FPGA literature [3].

6. **Expanded evaluation dimensions (power, security, and reliability).**
   Beyond latency/throughput/resource metrics, future extensions could incorporate power measurements and energy-per-inference evaluation, and also explore security-aware FPGA inference designs that leverage on-chip memories and architecture choices for remembered or precomputed components, as suggested in related FPGA CNN security/efficiency work [5].

In conclusion, this thesis demonstrates that fpgaConvNet provides a practical and effective toolflow for mapping CNN inference onto embedded FPGA platforms and for systematically exploring performance-resource trade-offs across CNN architectures and FPGA device configurations [1]. By combining real on-board deployment on the ZedBoard with broader evaluation using the KCU105 configuration, the work establishes a solid baseline for future research toward dynamic, adaptive, and context-aware CNN acceleration on reconfigurable hardware [2]–[4].

7. References

[1] I. Venieris, S. Kouris, and C.-S. Bouganis, "fpgaConvNet: A Framework for Mapping Convolutional Neural Networks on FPGAs," arXiv preprint arXiv:1711.08740, 2017. DOI: 10.48550/arXiv.1711.08740.

[2] M. Piponidis and T. Theocharides, "Dynamic CNN Parameter Exploration for Multi-Altitude UAV Object Detection," arXiv preprint arXiv:2410.14234, 2024.

[3] A. Dimitriou, B. Biggs, J. Hare, and G. V. Merrett, "FPGA Acceleration of Dynamic Neural Networks: Challenges and Advancements," arXiv preprint arXiv:2408.11743, 2024.

[4] F. Manca, F. Ratto, and F. Palumbo, "ONNX-to-Hardware Design Flow for Adaptive Neural-Network Inference on FPGAs," arXiv preprint arXiv:2402.11037, 2024.

[5] K. Guha and A. Chakrabarti, "Secure, Energy-Efficient Implementation of CNN on FPGAs for Accuracy Dependent Real Time Task Processing," in TENCON 2020 - 2020 IEEE Region 10 Conference (TENCON), 2020, doi: 10.1109/TENCON50793.2020.9293779.

[6] A. Muñío-Gracia, J. Fernández-Berni, R. Carmona-Galán, and Á. Rodríguez-Vázquez, "VersaTile: Convolutional Neural Network Mapping on FPGAs," arXiv preprint arXiv:2006.08711, 2020.

[7] A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," University of Toronto, Technical Report, 2009.

[8] Y. Chen (chenyaofo), "Pretrained models on CIFAR10/100 in PyTorch (pytorch-cifar-models)," GitHub repository. Available: https://github.com/chenyaofo/pytorch-cifar-models. Accessed: Dec. 13, 2025.

[9] A. Montgomerie, "fpgaConvNet Tutorials (fpgaconvnet-tutorial)," GitHub repository. Available: https://github.com/AlexMontgomerie/fpgaconvnet-tutorial. Accessed: Dec. 13, 2025.

[10] AMD/Xilinx, "Vitis™ AI 3.5 Documentation," online documentation. Available: https://xilinx.github.io/Vitis-AI/3.5/html/index.html. Accessed: Dec. 13, 2025.

[11] Microsoft, "ONNX Runtime Documentation," online documentation. Available: https://onnxruntime.ai/docs/. Accessed: Dec. 13, 2025.

[12] PyTorch, "PyTorch → ONNX Export Documentation," online documentation. Available: https://docs.pytorch.org/docs/stable/onnx.html. Accessed: Dec. 13, 2025.

[13] PyTorch, "Export a PyTorch model to ONNX (tutorial)," online tutorial. Available: https://docs.pytorch.org/tutorials/beginner/onnx/export_simple_model_to_onnx_tutorial.html. Accessed: Dec. 13, 2025.

[14] AMD/Xilinx, "Vivado Design Suite User Guide: Release Notes, Installation, and Licensing (UG973) — 2019.1," 2019. Available: https://docs.amd.com/v/u/2019.1-English/ug973-vivado-release-notes-install-license. Accessed: Dec. 13, 2025.

[15] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv preprint arXiv:1512.03385, 2015.

[16] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv preprint arXiv:1409.1556, 2014.

[17] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," arXiv preprint arXiv:1801.04381, 2018.

[18] N. Ma, X. Zhang, H.-T. Zheng, and J. Sun, "ShuffleNet V2: Practical Guidelines for Efficient CNN Architecture Design," arXiv preprint arXiv:1807.11164, 2018.

[19] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style ConvNets Great Again," arXiv preprint arXiv:2101.03697, 2021.

[20] Y. Umuroglu et al., "FINN: A Framework for Fast, Scalable Binarized Neural Network Inference," arXiv preprint arXiv:1612.07119, 2016.

[21] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST Database of handwritten digits," online resource. Available: http://yann.lecun.com/exdb/mnist/. Accessed: Dec. 13, 2025.

[22] AMD/Xilinx, "Vitis High-Level Synthesis User Guide," online documentation, docs.amd.com. Accessed: Dec. 12, 2025. Available: https://docs.amd.com/r/en-US/ug1399-vitis-hls.

[23] AMD/Xilinx, "Vitis AI Documentation — Supported Devices / Platforms," online documentation, docs.amd.com (or xilinx.github.io). Accessed: Dec. 12, 2025. Available: https://docs.amd.com/r/2023.1-English/ug1400-vitis-embedded/Supported-Platforms.